

3. this

- JavaScript의 this는 해당 함수 호출 패턴에 따라서 의미가 다르다.
- 전역 객체를 참조 (Global Scope)
- 메소드 내부의 this는 해당 메소드를 호출한 부모 객체를 참조
- 생성자 함수 코드 내부의 this는 새로 생성된 객체를 참조.
- call()과 apply() 메소드로 함수를 호출할 때, 함수의 this는 첫 번째 인자로 넘겨받은 객체를 참조
- 프로토타입 객체 메소드 내부의 this도 해당 메소드를 호출한 부모 객체를 참조.
- JavaScript의 this 키워드는 접근제어자 public 역할

this는 객체 자신의 프로퍼티나 메서드를 참조하기 위한 자기 참조 변수(self-referencing variable)다.

this가 가리키는 값, 즉 this 바인딩은 함수 호출 방식에 따라 동적으로 결정된다.

- 일반 함수 호출 : 전역 객체
- 메서드 호출 : 메서드를 호출한 객체 마침표 앞의 객체
- 생성자 함수로서 호출 : 생성자 함수가 생성할 인스턴스

1. Global Scope

- JavaScript의 모든 전역 변수는 실제로 전역 객체(브라우저에서는 window객체)의 프로퍼티들입니다.
- 모든 프로그램은 global code의 실행에 의해서 시작하고 this는 정해진 실행 문맥의 안에 고정되기 때문에 JavaScript의 this는 global code의 전역 객체입니다.

2. 부모 객체 참조

Counter 객체가 increment 메소드 호출 (this 의 범위는 자신을 호출한 객체 참조)

{% code lineNumbers="true" %}

```
let varValue = 100;
const counter = {
  varValue : 0,
  increment: function() {
    console.log(this);
    this.varValue += 1;
  }
};

counter.increment();
console.log(counter.varValue ); // 1

// 전역화
const inc = counter.increment;
inc();
console.log(varValue); // 101
```

{% endcode %}

- 10 line : counter 객체의 increment 메소드 호출 하면 4 line의 this는 counter 입니다.
- 13 line : counter.increment를 inc 변수에 담아서 실행하고 있으므로 4 line의 this는 window 입니다.

3. 내부 객체 this

JavaScript에서는 내부 함수 호출 패턴을 정의해 놓지 않기 때문

- 내부 함수도 결국 함수이므로 이를 호출할 때는 함수 호출로 취급 됨
- 함수 호출 패턴 규칙에 따라 내부 함수의 this는 전역 객체를 참조
- func1(메소드)에서는 메소드 내부의 val 값이 출력,
- func2(내부 함수), func3(내부 함수)에서는 전역 변수의 val 값이 출력

{% code lineNumbers="true" %}

```
val = 100;
var thisScope = {
  val: 1,
  func1: function() {
    console.log("func1() :: ", this);
    this.val += 1;
    console.log('func1() this.val :: ', this.val); // func1() this.va
l: 2
  }
  func2 = function() {
    console.log("func2() :: ", this);
    this.val += 1;
    console.log('func2() this.val: ' + this.val); // func2() this.v
al: 101
  }
  func3 = function() {
    console.log("func3() :: ", this);
    this.val += 1;
    console.log('func3() this.val: ' + this.val);
    // func3() this.val: 102
  }
  func3();
}
func2();
};

thisScope.func1();
```

{% endcode %}

- 24 line : thisScope.func1() 호출
- 5 line : thisScope 객체에 정의된 func1이므로 this 는 counter 객체를 가리킵니다.

그러므로 10 line, 11 line은 thisScope 의 val 변수를 참조하여 계산 됩니다.

- 8 line : func2() 호출
- 9 line : thisScope 객체 생성시 func1 객체는 있지 않습니다. 즉 func2 객체는 global scope 에 있어서 9 line의 this는 window 객체 입니다.
\\
- 18 line : func2와 같은 원리로 13 line의 this는 global scope 에 있어서 9 line의 this는 window 객체 입니다.
\\

4. This – 생성자 함수 호출

생성자 함수를 호출할 때, 생성자 함수 코드 내부의 this는 새로 생성된 객체를 참조합니다.

```
{% code lineNumbers="true" %}
```

```
function F(v) {
  console.log(this); // F {}
  this.fVal= v;
}
var f = new F("constructor function");
console.log(f);      // F { val: 'constructor function' }
console.log(f.fVa);  // constructor function
console.log(fVa);    // Uncaught ReferenceError: fVa is not define
```

```
{% endcode %}
```

- 5 line : new 생성자를 통해서 객체 생성
- 2 line : new 로 생성자 객체 이므로 this는 F 객체를 가리킵니다.
- 8 line : val 는 global 영역에 선언이 되어 있지 않아서 Uncaught ReferenceError: fVa is not define 오류가 발생합니다.

4-1 call과 apply Method 함수 호출

- JavaScript에는 이러한 내부적인 this 바인딩 이외에도 this를 특정 객체에 명시적으로 바인딩 시키는 apply()와 call() 메소드 제공.
- apply()과 call() 메소드로 함수를 호출할 때, 함수는 첫 번째 인자로 넘겨받은 객체를 this로 바인딩하고, 두 번째 인자로 넘긴 값들은 함수의 인자로 전달
- apply() 메소드는 배열 형태로 인자를 넘기고, call() 메소드는 배열 형태로 넘긴 것을 각각 하나의 인자로 넘기는 것입니다.

```
{% code lineNumbers="true" %}
```

```
const fnAdd = function(x, y) {
  console.log("x ::" , x);
  console.log("y ::" , y);
  console.log( this );
}
```

```

    console.log("this.val :: ", this.val);
    // 첫번째 apply에서 호출 되는 경우 0으로 초기화 함
    // obj에 val이 정의 되어 있으므로 ) obj가 this
    this.val = x + y;
    this.val += this.hval;
};

const data = {
    val: 0,
    hval : 10
};

fnAdd.apply(data , [2, 8]);

console.log(data.val); // 20
console.log("=====");
fnAdd.call(data, 2, 8);
console.log(data.val); // 20
console.log(data);

```

{% endcode %}

- 17 line : apply를 사용하여 fnAdd 객체를 사용하고 있으므로
 - 2 line : 2
 - 3 line : 8
 - 4 line : this 는 data로 {val: 0, hval: 10}
 - 5 line : this.val 은 0
- 19 line : 8 line, 9 line의 결과로 20
- 21 line : call로 fnAdd 객체를 사용하고 있으므로
 - 2 line : 2
 - 3 line : 8
 - 4 line : this 는 data로 {val: 20, hval: 10}
 - 5 line : this.val 은 20
- 12 line : 8 line, 9 line의 결과로 20
- 23 line : {val: 20, hval: 10}

5. This - 프로토타입

프로토타입 객체 메소드의 this로 해당 메서드 호출

- 프로토타입 객체는 메소드를 가질 수 있다.
- 이 프로토타입 메소드 내부에서의 this도 똑같이 메소드 내부의 this는 해당 메소드를 호출한 부모 객체를 참조.

getName() 메소드는 foo 객체에서 찾을 수 없으므로 프로토타입 체이닝이 발생 fPerson 객체의 프로토타입 객체인 person.prototype에서 getName() 메소드가 있으므로, 이 메소드가 호출 getName() 메소드를 호출한 객체는 fPerson 이므로, this는 foo 객체에 바인딩 fPerson.getName()의 결과로 홍길동 가 출력

{% code lineNumbers="true" %}

```
function Person(name) {
  console.log(" Person This :: " , this);
  this.name = name;
}

Person.prototype.getName = function() {
  console.log("This :: " , this);
  return this.name;
}

const fPerson = new Person("홍길동"); // this :: Person {}
console.log(fPerson.getName());
// this :: Person {name: '홍길동'} -> 체이닝 발생 :: 홍길동

Person.prototype.name = "홍당무";
console.log(fPerson.getName());
// this :: Person {name: '홍길동'} -> 체이닝 발생 :: 홍길동

console.log(Person.prototype.getName());
// this :: {name: '홍당무', getName: f, constructor: f} :: 홍당무
```

{% endcode %}

- 11 line : Person 객체 생성
 - 2 line : this -> Person {}
- 12 line : fPerson에는 getName() 이 없으므로 프로토타입 체이닝이 발생 하여 fPerson 객체의 프로토타입 객체인 person.prototype에서 getName() 메소드가 있으므로, 이 메소드가 호출하여this는 Person {name: '홍길동'} 객체 임
- 16 line : fPerson에는 getName() 이 없으므로 프로토타입 체이닝이 발생 하여 this는 Person {name: '홍길동'} 객체 임
- 19 line : {name: '홍당무', getName: f, constructor: f}

6. Public 역할

- public, private 키워드 자체를 지원하지 않지만 public 역할을 하는 this 키워드와 private 역할을 하는 var 키워드가 있습니다. JavaScript에서는 이 두 키워드를 사용하여 캡슐화

```
function Person(name, age) {
  let Name = name;
  this.Age = age;
  this.getName = function() {
    return Name;
  }
}

var fPerson = new Person('홍당무', 26);
console.log(fPerson.Age); // 26
console.log(fPerson.Name); // undefined 내부 변수로 접근 불가
console.log(fPerson.getName()); // 홍당무
```

7. This 함정

7-1. 내부 함수의 this는 global 입니다.

fTest 에서 this가 Func를 가리킬 것으로 생각되지만. 실제로는 그렇지 않다.

```
var Func = {
  name : "홍당무"
};

Func.method = function() {
  function fTest() {
    console.log("this= ", this); // Object [global] {.....
    console.log("this.name = ", this.name); // undefined
  };

  fTest();
}
Func.method();
```

fTest에서 Func에 접근하려면 method에 Local 변수를 하나 만들고 Func를 가리키게 하여야 한다

```
var Func = {
  name : "홍당무"
};

Func.method = function() {
  let self = this; // Local 변수를 통한 접근
  function fTest() {
    console.log("self = ", self); // { method: [Function], name: '홍길
동' }
    console.log("self.name = ", self.name); // 홍길동
  };
  fTest();
}

Func.method();
```

7-2. 콜백 함수에 this에 다른 값을 전달 하는 방법

```
{% code lineNumbers="true" %}
```

```
var ObjA = {
  values: [1,2,3],
  fPrintValues: function(value, index) {
```

```

        var self = this;
        console.log(self);
        // { values: [ 1, 2, 3 ], fPrintValues: [Function: fPrintValues]
    }

    return function(value, index) {
        console.log(this); // global
        return console.log(self.values, value, index);
    }
}

var fCb = ObjA.fPrintValues();
console.log(`${fCb}`);
fCb(10,10);
[10,20,30].forEach(fCb);

```

{% endcode %}

- 14 line : ObjA.fPrintValues() 를 fCb 에 할당 (callback 함수)
 - 5 line : {values: Array(3), fPrintValues: f}
- 15 line : callback 함수 출력
- 16 line : 호출
 - 8 line : this 는 global
 - 9 line : [1, 2, 3] 10 10
- 17 line : for - this 는 global
 - [1, 2, 3] 10 0
 - [1, 2, 3] 20 1
 - [1, 2, 3] 30 2

7-3. ECMAScript 5 이후 this

ECMAScript 5부터는 익명 함수와 결합된 bind 메서드를 사용하여 같은 결과를 얻을 수 있다.

```

var Func = {
    name : "홍당무"
};

Func.method = function() {
    var ftest = function () {
        console.log("this = ", this);
        console.log("this.name = ", this.name);
    }.bind(this);

    ftest();
}

Func.method();

```

```
var objA = {
  name: 'OBJ A',
  func: function() {
    console.log(this.name);
  }
}
objA.func(); // OBJ A
setTimeout(objA.func, 100); // undefined

setTimeout(objA.func.bind(objA), 100); // OBJ A

var objB = { name: 'OBJ B' };
setTimeout(objA.func.bind(objB), 100); // OBJ B
```

7-4. 프로토 타입 상속 (prototypal inheritance)

```
function Func01() {};
Func01.prototype.name = "홀길동";
Func01.prototype.method = function() {
  console.log("this ::", this);
  console.log("this.name ::", this.name);
};

new Func01().method(); // this :: Func01 {}, this.name :: 홀길동

function Func02() {}
Func02.prototype = Func01.prototype; // 프로토 타입 상속

var fFunc02 = new Func02();
fFunc02.method(); // this :: Func01 {}, this.name :: 홀길동
```